```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Airline_Flight_Performance_Dataset.csv to Airline_Flight_Perforr
```

Double-click (or enter) to edit

```
import pandas as pd

# Load the uploaded CSV file into a DataFrame
df = pd.read_csv('Airline_Flight_Performance_Dataset.csv')
display(df)
```

	Flight_ID	Flight_Date	Airline	Origin	Origin_Code	Destination
0	FL10002	2026-05-21	Air India Express	Mumbai	BOM	Delhi
1	FL10003	2026-04-23	Air India Express	Goa	GOI	Pune
2	FL10004	2026-01-23	Vistara	Lucknow	LKO	Chennai
3	FL10005	2026-08-21	IndiGo	Pune	PNQ	Chennai
4	FL10006	2026-04-28	Air India	Kolkata	CCU	Delhi
...	...	...	...	...	...	...
495	FL10497	2026-06-18	Akasa Air	Pune	PNQ	Mumbai
496	FL10498	2026-02-18	Vistara	Lucknow	LKO	Ahmedabad
497	FL10499	2026-09-02	Akasa Air	Goa	GOI	Pune
498	FL10500	2026-04-04	Akasa Air	Pune	PNQ	Goa
499	FL10501	2026-05-03	IndiGo	Mumbai	BOM	Bengaluru

500 rows × 27 columns

Double-click (or enter) to edit

```
!pip -q install plotly dash
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.9/8.9 MB 58.7 MB/
```

```
import pandas as pd
import numpy as np

df = pd.read_csv("Airline_Flight_Performance_Dataset.csv")

df["Flight_Date"] = pd.to_datetime(df["Flight_Date"])

df.head()
```

	Flight_ID	Flight_Date	Airline	Origin	Origin_Code	Destination	D
0	FL10002	2026-05-21	Air India Express	Mumbai	BOM	Delhi	
1	FL10003	2026-04-23	Air India Express	Goa	GOI	Pune	
2	FL10004	2026-01-23	Vistara	Lucknow	LKO	Chennai	
3	FL10005	2026-08-21	IndiGo	Pune	PNQ	Chennai	
4	FL10006	2026-04-28	Air India	Kolkata	CCU	Delhi	

5 rows × 27 columns

```
print("Total Flights:", len(df))
print("Total Revenue: ₹", f"{df['Revenue_INR'].sum():,.0f}")
print("Total Profit: ₹", f"{df['Profit_INR'].sum():,.0f}")
print("Average Delay:", round(df["Delay_Minutes"].mean(), 2), "minutes")
print("Average Load Factor:", round(df["Load_Factor_Pct"].mean(), 2), "%")
```

```
Total Flights: 500
Total Revenue: ₹ 694,870,907
Total Profit: ₹ 262,714,951
Average Delay: 25.49 minutes
Average Load Factor: 80.6 %
```

```
import plotly.express as px
import plotly.graph_objects as go
from IPython.display import display, HTML

# ----------------------------
# KPI calculations
# ----------------------------
total_flights = len(df)
total_passengers = df["Passengers"].sum()
total_revenue = df["Revenue_INR"].sum()
total_profit = df["Profit_INR"].sum()

on_time_pct = (
```

```
    (df["Flight_Status"] == "On Time").sum() / total_flights * 100
)

avg_delay = df["Delay_Minutes"].mean()
avg_load_factor = df["Load_Factor_Pct"].mean()
avg_rating = df["Customer_Rating"].mean()

# ----------------------------
# KPI cards
# ----------------------------
kpi_html = f"""
<div style="display:flex; gap:15px; flex-wrap:wrap; margin-bottom:25px;">

<div style="background:#1f2937;color:white;padding:20px;border-radius:12px;
width:200px;text-align:center;">
<h4>Total Flights</h4>
<h2>{total_flights:,}</h2>
</div>

<div style="background:#374151;color:white;padding:20px;border-radius:12px;
width:200px;text-align:center;">
<h4>Passengers</h4>
<h2>{total_passengers:,}</h2>
</div>

<div style="background:#4b5563;color:white;padding:20px;border-radius:12px;
width:200px;text-align:center;">
<h4>Revenue</h4>
<h2>₹{total_revenue/10000000:.2f} Cr</h2>
</div>

<div style="background:#6b7280;color:white;padding:20px;border-radius:12px;
width:200px;text-align:center;">
<h4>Profit</h4>
<h2>₹{total_profit/10000000:.2f} Cr</h2>
</div>

<div style="background:#111827;color:white;padding:20px;border-radius:12px;
width:200px;text-align:center;">
<h4>On-Time Rate</h4>
<h2>{on_time_pct:.1f}%</h2>
</div>

</div>
"""

display(HTML("""
<h1 style="text-align:center;">
✈️ Airline Flight Performance Dashboard
</h1>
"""))

display(HTML(kpi_html))

# ----------------------------
# Chart 1: Flights by Airline
# ----------------------------
airline_count = (
    df.groupby("Airline")
    .size()
    .reset_index(name="Flights")
    .sort_values("Flights", ascending=False)
)
```

```
fig1 = px.bar(
    airline_count,
    x="Airline",
    y="Flights",
    title="Flights by Airline",
    text="Flights"
)

fig1.update_layout(template="plotly_white")
fig1.show()

# ----------------------------
# Chart 2: Flight Status
# ----------------------------
status_count = (
    df["Flight_Status"]
    .value_counts()
    .reset_index()
)
status_count.columns = ["Flight_Status", "Flights"]

fig2 = px.pie(
    status_count,
    names="Flight_Status",
    values="Flights",
    title="Flight Status Distribution",
    hole=0.4
)
fig2.show()

# ----------------------------
# Chart 3: Revenue by Airline
# ----------------------------
revenue_airline = (
    df.groupby("Airline")["Revenue_INR"]
    .sum()
    .reset_index()
)

fig3 = px.bar(
    revenue_airline,
    x="Airline",
    y="Revenue_INR",
    title="Revenue by Airline",
    text_auto=".2s"
)
fig3.update_layout(
    template="plotly_white",
    yaxis_title="Revenue (INR)"
)
fig3.show()

# ----------------------------
# Chart 4: Profit by Airline
# ----------------------------
profit_airline = (
    df.groupby("Airline")["Profit_INR"]
    .sum()
    .reset_index()
)

fig4 = px.bar(
    profit_airline,
    x="Airline",
```

```
    y="Profit_INR",
    title="Profit by Airline",
    text_auto=".2s"
)
fig4.update_layout(
    template="plotly_white",
    yaxis_title="Profit (INR)"
)
fig4.show()

# ----------------------------
# Chart 5: Average Delay
# ----------------------------
delay_airline = (
    df.groupby("Airline")["Delay_Minutes"]
    .mean()
    .reset_index()
)

fig5 = px.bar(
    delay_airline,
    x="Airline",
    y="Delay_Minutes",
    title="Average Flight Delay by Airline",
    text_auto=".1f"
)
fig5.update_layout(
    template="plotly_white",
    yaxis_title="Average Delay (Minutes)"
)
fig5.show()

# ----------------------------
# Chart 6: Load Factor
# ----------------------------
load_airline = (
    df.groupby("Airline")["Load_Factor_Pct"]
    .mean()
    .reset_index()
)

fig6 = px.bar(
    load_airline,
    x="Airline",
    y="Load_Factor_Pct",
    title="Average Load Factor by Airline",
    text_auto=".1f"
)
fig6.update_layout(
    template="plotly_white",
    yaxis_title="Load Factor (%)"
)
fig6.show()

# ----------------------------
# Chart 7: Monthly Flight Trend
# ----------------------------
# Ensure Flight_Date is converted to datetime object to use .dt accessor
df["Flight_Date"] = pd.to_datetime(df["Flight_Date"])
df["Month"] = df["Flight_Date"].dt.to_period("M").astype(str)

monthly = (
    df.groupby("Month")
    .size()
```

```
    .reset_index(name="Flights")
)

fig7 = px.line(
    monthly,
    x="Month",
    y="Flights",
    markers=True,
    title="Monthly Flight Trend"
)
fig7.update_layout(template="plotly_white")
fig7.show()

# ----------------------------
# Chart 8: Distance vs Revenue
# ----------------------------
fig8 = px.scatter(
    df,
    x="Distance_KM",
    y="Revenue_INR",
    color="Airline",
    title="Distance vs Revenue",
    hover_data=[
        "Flight_ID",
        "Origin",
        "Destination",
        "Passengers"
    ]
)
fig8.show()
```

✈️ Airline Flight Performance Dashboard

Total Flights

500

Passengers

80,549

Revenue

₹69.49 Cr

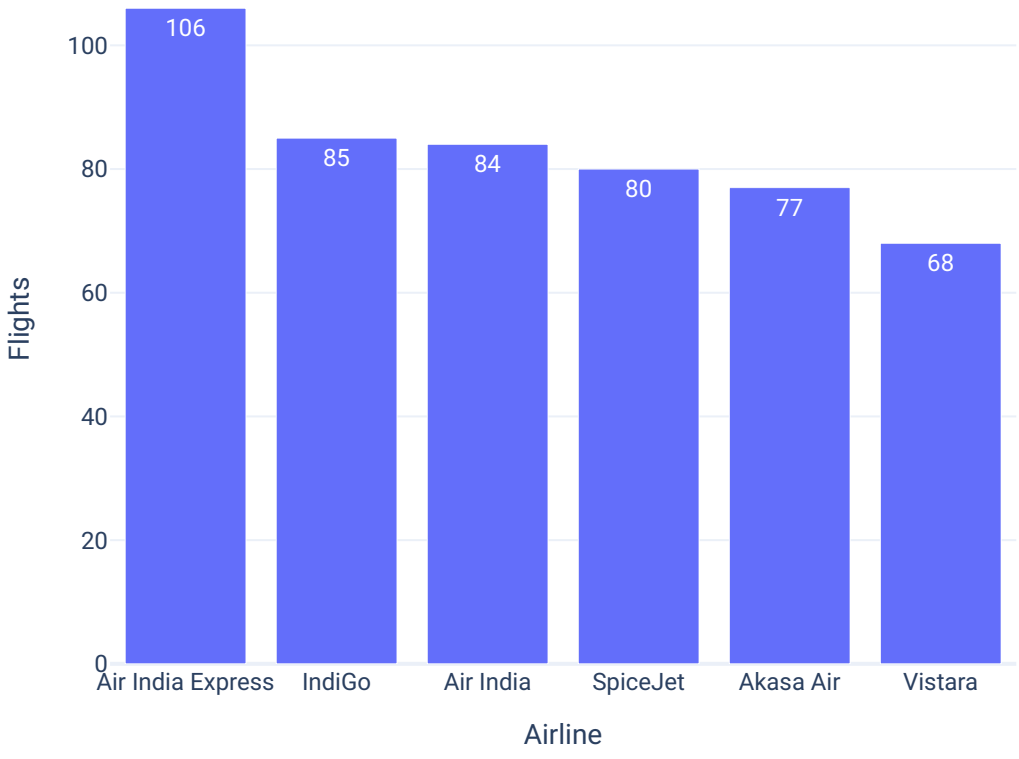
Profit

₹26.27 Cr

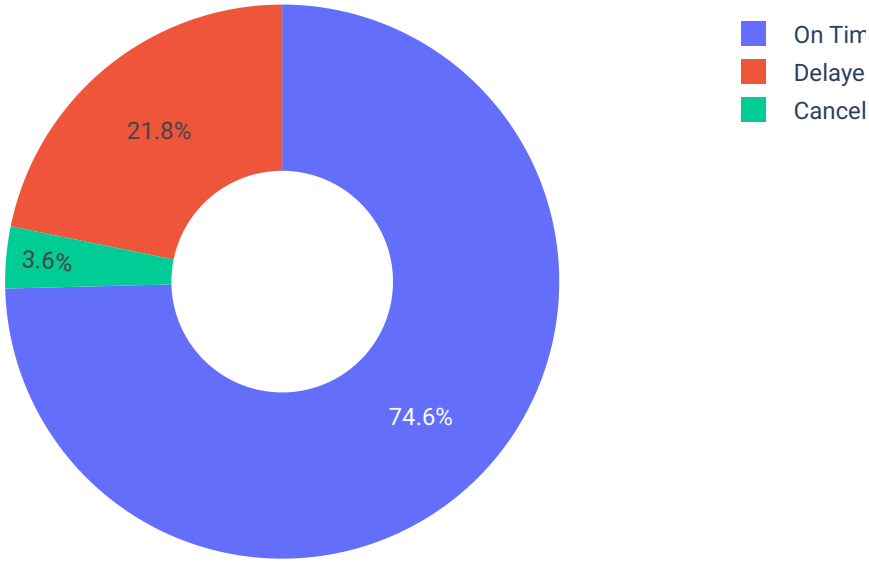
On-Time Rate

74.6%

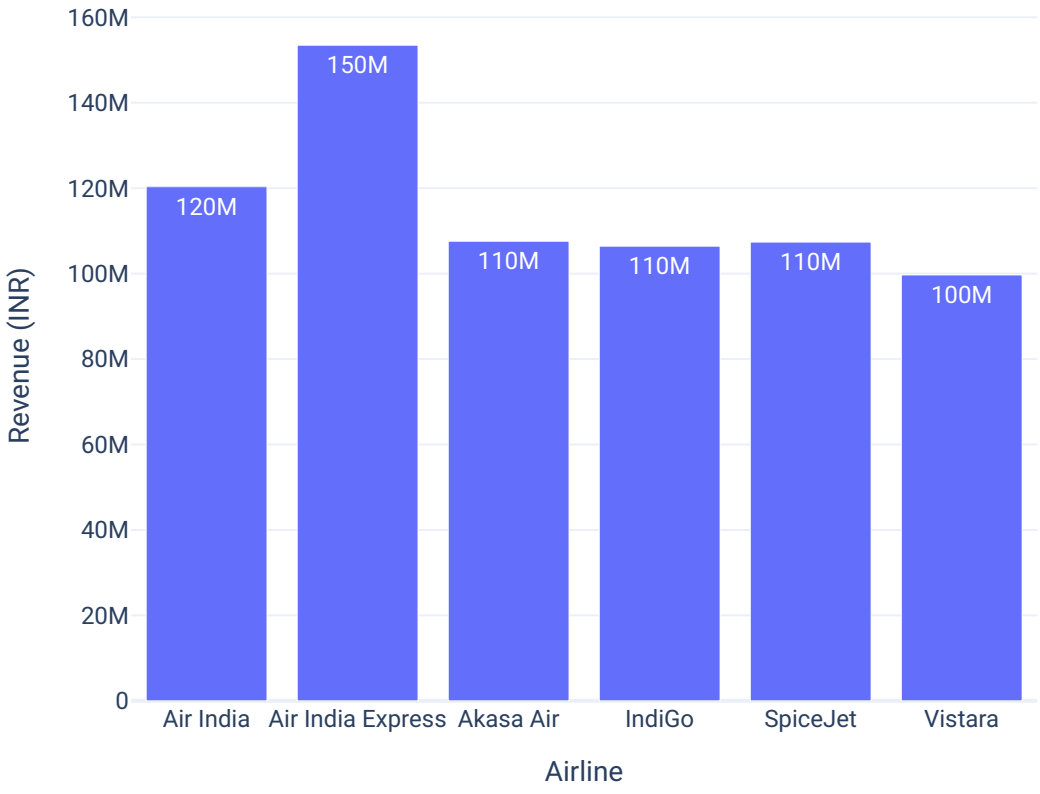
Flights by Airline



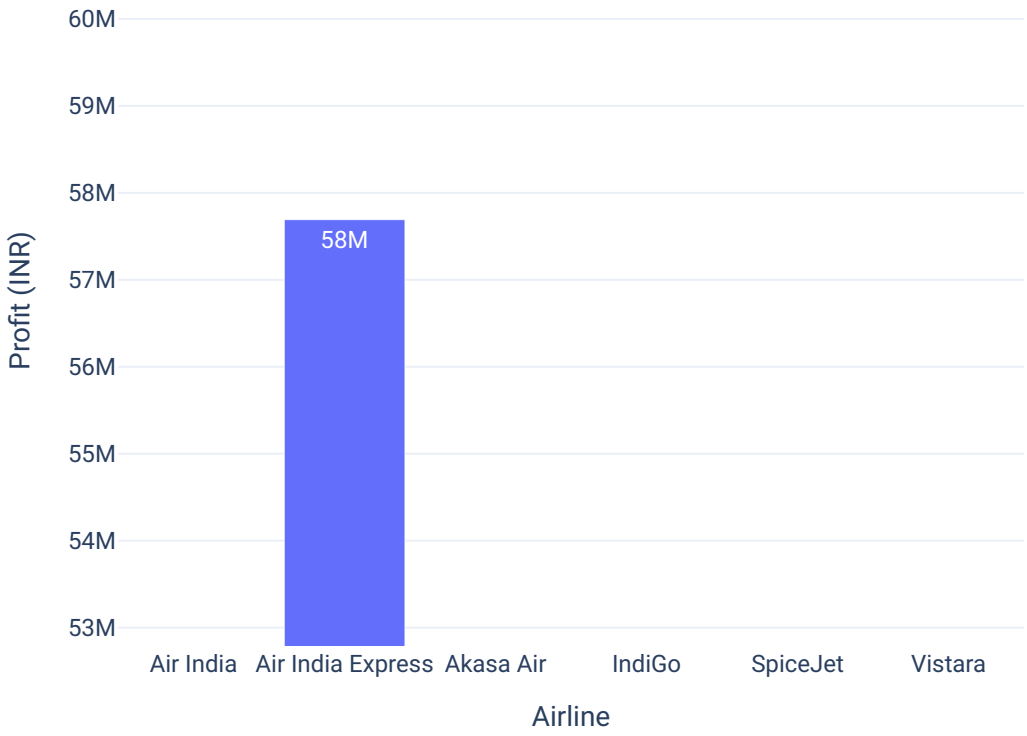
Flight Status Distribution



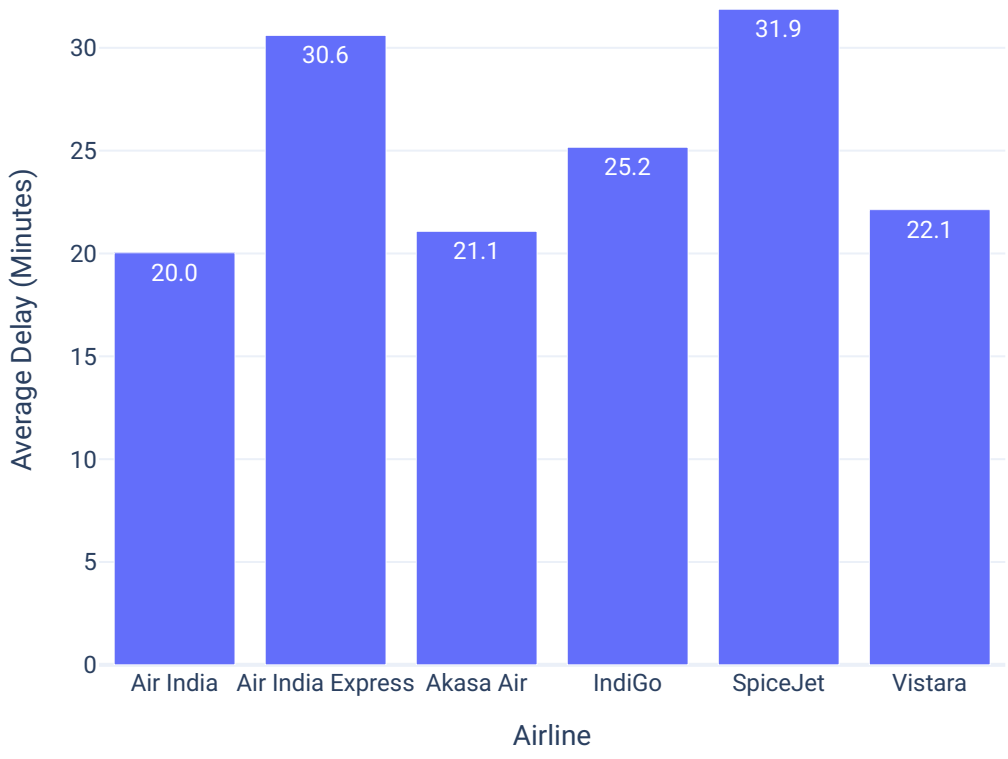
Revenue by Airline



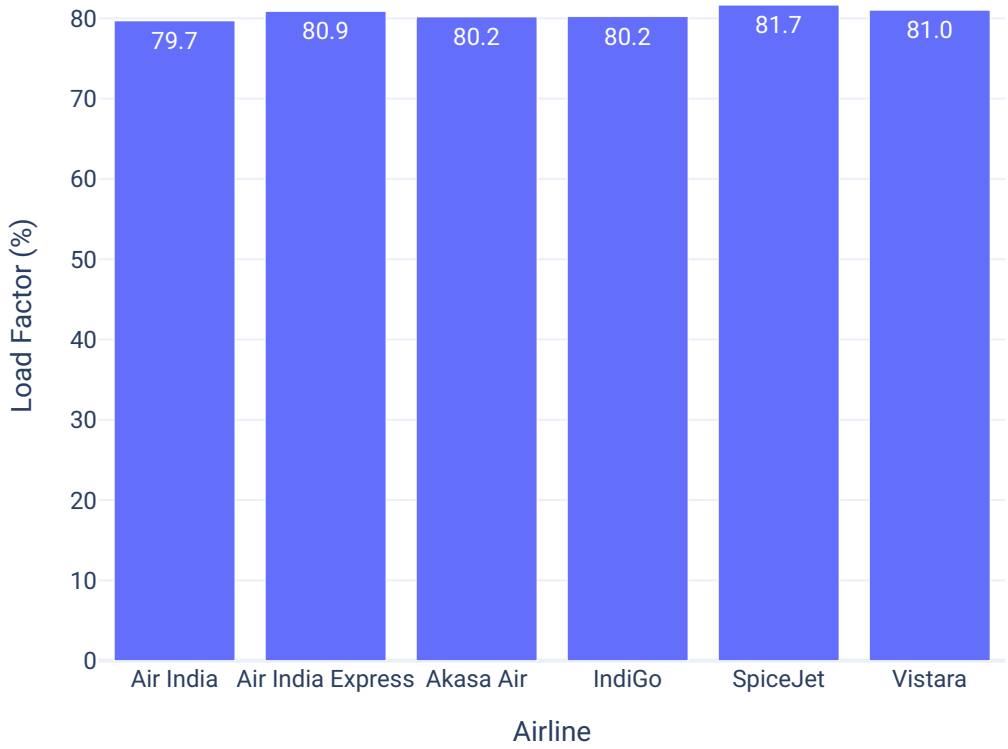
Profit by Airline



Average Flight Delay by Airline



Average Load Factor by Airline



Monthly Flight Trend



```
!pip -q install dash jupyter-dash plotly
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.9/8.9 MB 64.2 MB/
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 101.2 MB
```

Gemini

```
jupyter_dash import
dash import dcc, htm
t os
t pandas as pd
t plotly.express as

 JupyterDash(__name_
dash import Dash, dc

ely load df if it is

f
t NameError:
f os.path.exists("Ai
   df = pd.read_csv(
   df["Flight_Date"]
lse:
   # Safe empty fall
   df = pd.DataFrame

ern Dash setup (repl
 Dash(__name__)

nes = sorted(df["Air
ses = sorted(df["Fli
ns = sorted(df["Orig
nations = sorted(df[
nes = sorted(df["Air
ses = sorted(df["Fli
ns = sorted(df["Orig
nations = sorted(df[

ayout = html.Div(
tyle={
   "fontFamily": "Ar
   "padding": "25px"
   "backgroundColor"
,
hildren=[

   html.H1(
       "✈️ Airline F
       style={"textA
   ),

   html.P(
       "Interactive
       style={"textA
```

```
    ),

    # FILTERS
    html.Div([


        html.Div([
            html.Labe
            dcc.Dropd
                id="a
                optio
                value
                multi
            )
        ], style={"wi

        html.Div([
            html.Labe
            dcc.Dropd
                id="s
                optio
                value
                multi
            )
        ], style={"wi

        html.Div([
            html.Labe
            dcc.Dropd
                id="o
                optio
                value
                multi
            )
        ], style={"wi

        html.Div([
            html.Labe
            dcc.Dropd
                id="d
                optio
                value
                multi
            )
        ], style={"wi

    ]),

    html.Br(),

    # KPI CARDS
    html.Div(id="kpi_

    # CHARTS
    html.Div([
```

```
        dcc.Graph(id=
        dcc.Graph(id=
    ], style={"displa

    html.Div([
        dcc.Graph(id=
        dcc.Graph(id=
    ], style={"displa

    html.Div([
        dcc.Graph(id=
        dcc.Graph(id=
    ], style={"displa

    dcc.Graph(id="mon

    dcc.Graph(id="sca



callback(

    Output("kpi_cards
    Output("airline_c
    Output("status_ch
    Output("revenue_c
    Output("profit_ch
    Output("delay_cha
    Output("load_char
    Output("monthly_c
    Output("scatter_c
,

    Input("airline_fi
    Input("status_fil
    Input("origin_fil
    Input("destinatio


pdate_dashboard(
elected_airlines,
elected_status,
elected_origins,
elected_destinations

f df.empty or not se
    empty_fig = px.sc
    return html.Div("

iltered = df[
    df["Airline"].isi
    df["Flight_Status
    df["Origin"].isin
    df["Destination"]
```

```
 KPIs
otal_flights = len(f
assengers = filtered
evenue = filtered["R
rofit = filtered["Pr

n_time = (
   (filtered["Flight
   if len(filtered)


ards = html.Div(
   [
       html.Div([
           html.H4("
           html.H2(f
       ], className=

       html.Div([
           html.H4("
           html.H2(f
       ], className=

       html.Div([
           html.H4("
           html.H2(f
       ], className=

       html.Div([
           html.H4("
           html.H2(f
       ], className=

       html.Div([
           html.H4("
           html.H2(f
       ], className=
   ],
   style={
       "display": "f
       "gap": "15px"
       "marginBottom
   }


 Airline flights
irline_data = (
   filtered.groupby(
   .size()
   .reset_index(name


ig_airline = px.bar(
```

```
    airline_data,
    x="Airline",
    y="Flights",
    title="Flights by
    text="Flights"


 Status
tatus_data = (
    filtered["Flight_
    .value_counts()
    .reset_index()


tatus_data.columns =

ig_status = px.pie(
    status_data,
    names="Flight_Sta
    values="Flights",
    title="Flight Sta
    hole=0.4


 Revenue
evenue_data = (
    filtered.groupby(
    .sum()
    .reset_index()


ig_revenue = px.bar(
    revenue_data,
    x="Airline",
    y="Revenue_INR",
    title="Revenue by
    text_auto=".2s"


 Profit
rofit_data = (
    filtered.groupby(
    .sum()
    .reset_index()


ig_profit = px.bar(
    profit_data,
    x="Airline",
    y="Profit_INR",
    title="Profit by
    text_auto=".2s"


 Delay
```

```
elay_data = (
   filtered.groupby(
   .mean()
   .reset_index()


ig_delay = px.bar(
   delay_data,
   x="Airline",
   y="Delay_Minutes"
   title="Average De
   text_auto=".1f"


 Load factor
oad_data = (
   filtered.groupby(
   .mean()
   .reset_index()


ig_load = px.bar(
   load_data,
   x="Airline",
   y="Load_Factor_Pc
   title="Average Lo
   text_auto=".1f"


 Monthly trend
iltered = filtered.c
iltered["Month"] = f

onthly = (
   filtered.groupby(
   .size()
   .reset_index(name


ig_monthly = px.line
   monthly,
   x="Month",
   y="Flights",
   markers=True,
   title="Monthly Fl

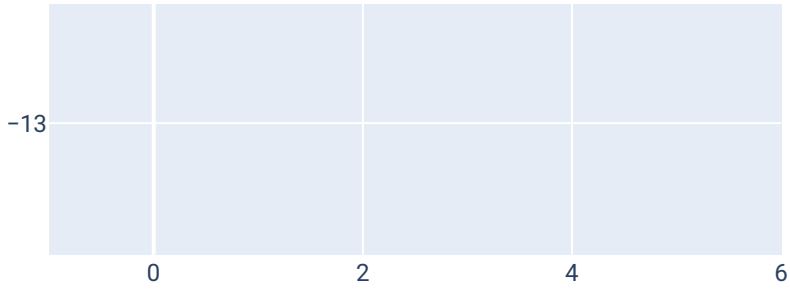

 Distance vs revenue
ig_scatter = px.scat
   filtered,
   x="Distance_KM",
   y="Revenue_INR",
   color="Airline",
   hover_data=[
       "Flight_ID",
       "Origin"
```

```
        "Origin",
        "Destination"
        "Passengers"
    ],
    title="Distance v


eturn (
    cards,
    fig_airline,
    fig_status,
    fig_revenue,
    fig_profit,
    fig_delay,
    fig_load,
    fig_monthly,
    fig_scatter

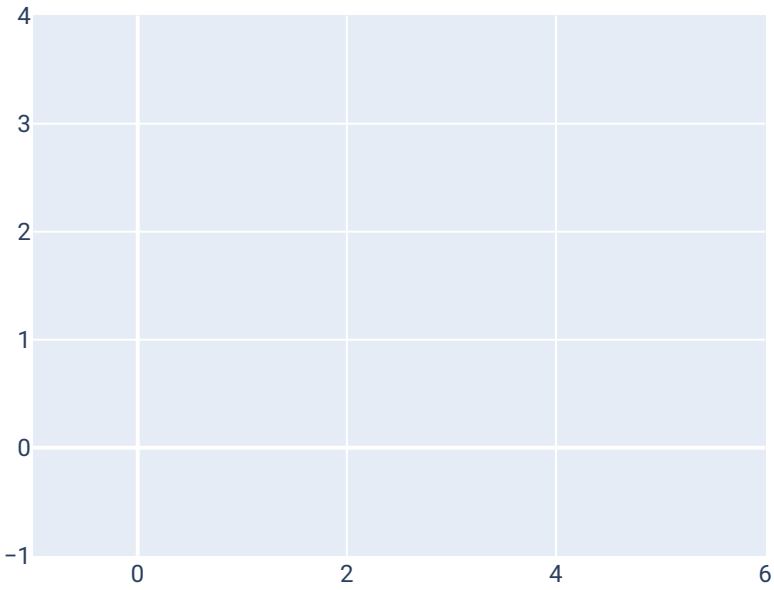


un(
ode="inline",
upyter_mode="inline",
ort=8050,
ebug=False
```





No Data Selected / Available




```
!pip -q install plotly ipywidgets
```

```
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, clear_output

air = widgets.Dropdown(
    options=["All"] + sorted(df.Airline.unique()),
    description="Airline:"
)

status = widgets.Dropdown(
    options=["All"] + sorted(df.Flight_Status.unique()),
    description="Status:"
)

out = widgets.Output()

def dashboard(*args):
    with out:
```

```
            clear_output(wait=True)

            d = df.copy()

            if air.value != "All":
                d = d[d.Airline == air.value]

            if status.value != "All":
                d = d[d.Flight_Status == status.value]

            print("✈️ AIRLINE FLIGHT PERFORMANCE DASHBOARD")
            print("Flights:", len(d))
            print("Passengers:", f"{d.Passengers.sum():,}")
            print("Revenue: ₹", f"{d.Revenue_INR.sum():,.0f}")
            print("Profit: ₹", f"{d.Profit_INR.sum():,.0f}")
            print("On-Time Rate:",
                  f"{(d.Flight_Status.eq('On Time').mean()*100):.1f}%")

            px.bar(
                d.groupby("Airline").size().reset_index(name="Flights"),
                x="Airline", y="Flights",
                title="Flights by Airline"
            ).show()

            px.pie(
                d, names="Flight_Status",
                title="Flight Status"
            ).show()

            px.bar(
                d.groupby("Airline")["Revenue_INR"].sum().reset_index(),
                x="Airline", y="Revenue_INR",
                title="Revenue by Airline"
            ).show()

            px.scatter(
                d, x="Distance_KM", y="Revenue_INR",
                color="Airline",
                title="Distance vs Revenue"
            ).show()

    air.observe(dashboard, names="value")
    status.observe(dashboard, names="value")

    display(air, status, out)
    dashboard()
```